Computer Graphics

Rasterization

Scan Conversion

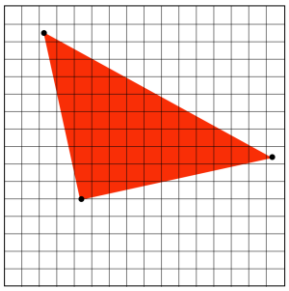
Dr. Hardik Jain
Assistant Professor
Department of Computer Science and Engineering
IIT Jodhpur

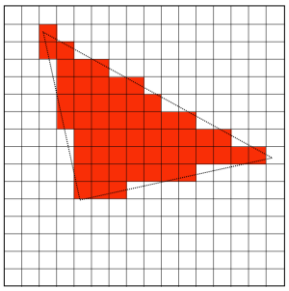
Few Slides are taken from CG lecture slides of Dr. Avinash Sharma

1

Rasterization or Scan Conversion

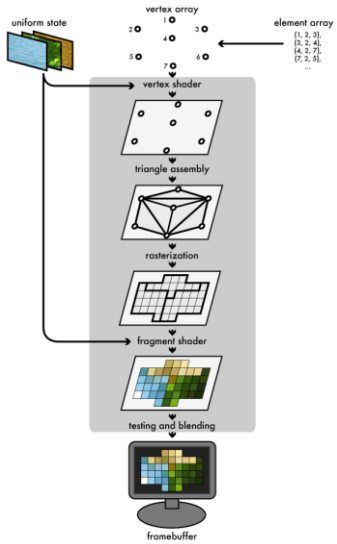
Breakdown into fragments





2

The (real-time) graphics pipeline



Slides of Dr. Rahul Narain

Even for 3D graphics,

1. First we project the vertices of each 3D triangle to their 2D locations on the screen
2. Then we rasterize the 2D triangle!

So it makes sense to study rasterization of 2D graphics.

Rasterization

- **Definition**: A process that converts continuous 3D geometric primitive with attributes to discrete pixel grid on 2D images
- Once the unnecessary primitives are removed (clipped), the primitives inside the viewing window (screen space) are rasterized into fragments.
- The rasterizer checks which pixels are covered or intersected by the projected primitive.
- Hence, the primitives are broken down into fragments or pixels to be projected onto the 2D screen.
- Usually this is performed by the graphics processing unit (GPU).

Scan Converting a Point

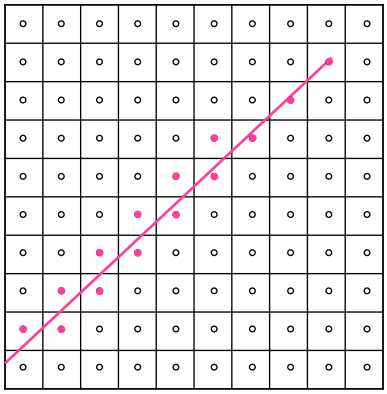
- The 3D point is transformed to its screen coordinates (u, v)
- Current colour of the 3D point is defined or known.
- Round the coordinates to frame buffer array indices (i, j).
- Frame buffer array is initialized to the background colour.
- Frame buffer at (i, j) is updated with the current Colour
- If `PointSize > 1`, assign the colour to a number of points in the neighbourhood!

5

Drawing Lines

Rasterization

- A continuous line (after clipping) will lie in the screen space
- Line needs to be rasterized to the pixel grid of the window.
- Find pixels that lie closest to the line.
- Each of these pixel needs to be given a color.



6

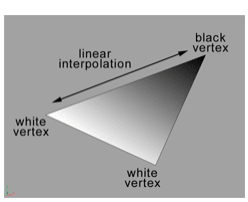
What about the attributes on the Line

We only know attributes on the end points of the line

7

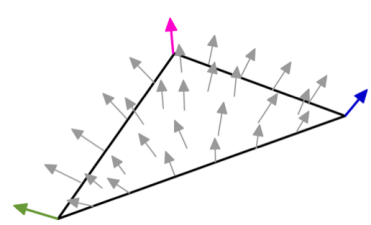
Linear Interpolation of Properties

Intensity Interpolation



- Each pixel needs: colour, depth, and texture coordinate.
- **Assumption**: Properties vary linearly across the plane.
- If we know the colour, texture coordinate, and depth at the vertices of the polygon (or line), these can be interpolated to pixels on the inside linearly!

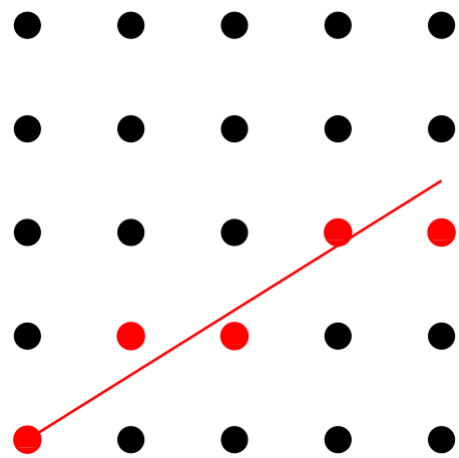
Normal Interpolation



8

Scan Converting a Line

- Identify the grid-points that lie on the line and colour them.
- Problem: Given two end-points on the grid, find the pixels on the line connecting them.
  - Incremental algorithm or Digital Differential Analyzer (DDA) algorithm.
  - Mid-Point Algorithm



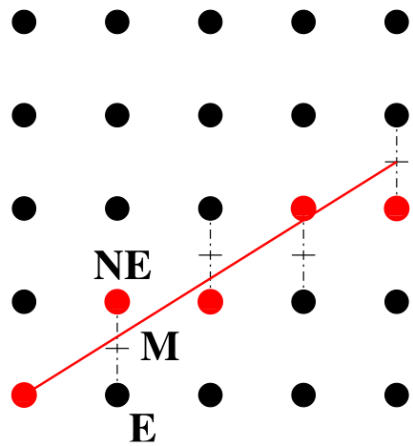
Incremental Line Drawing Algorithm

- If abs(slope) > 1, step through y values, adding inverse slopes to x at each step.
- Simple algorithm, easy to implement.
- Floating point calculations were expensive once!
- Can we do with integer arithmetic only?
  - Bresenham's Algorithm
  - Mid-Point Line Algorithm

Digital Differential Analyser (**DDA**) Algorithm

Function DrawLine(x_1, y_1, x_2, y_2, **colour**)
    $\Delta x \leftarrow x_2 - x_1, \Delta y \leftarrow y_2 - y_1$, slope $\leftarrow \Delta y / \Delta x$
    $x \leftarrow x_1, y \leftarrow y_1$
    While ($x < x_2$)
        WritePixel (x, round(y), **colour**)
        $x \leftarrow x + 1,\ \ y \leftarrow y +$ slope
    EndWhile
    WritePixel (x_2, y_2, **colour**)
EndFunction

Two Options at Each Step !



Mid-Point Line Algorithm

Function DrawLine $(x_1, y_1, x_2, y_2,$ colour$)$
  $a \leftarrow (y_2 - y_1),\ b \leftarrow (x_1 - x_2),\ x \leftarrow x_1,\ y \leftarrow y_1$
  $d \leftarrow 2a + b,\ \Delta_E \leftarrow 2a,\ \Delta_{NE} \leftarrow 2(a + b)$
  While $(x < x_2)$
    WritePixel$(x, y,$ colour$)$
    if $(d < 0)$ // East
      $d \leftarrow d + \Delta_E,\ x \leftarrow x + 1$
    else // North-East
      $d \leftarrow d + \Delta_{NE},\ x \leftarrow x + 1,\ y \leftarrow y + 1$
  EndWhile
  WritePixel$(i, j,$ colour$)$
EndFunction

Mid-Point Line Algorithm: Example

Draw a line from (10,10) to (20,17)

- $F(x,y) = 7\,x - 10\,y + 30 \;\rightarrow\; a = 7, b = -10\,, \Delta_{\mathbf{E}} = 2 * 7 = 14, \Delta_{\mathrm{NE}} = -6$
- $d_0 \;= 2 \;*\; 7 - 10 = 4$
- $d_0 > 0$: **NE**$(10,10)$: $(11,11), d_1 = 4 + -6 = -2$
- $d_1 < 0$: **E**$(11,11)$: $(12,11), d_2 = -2 + 14 = 12$
- $d_2 > 0$: **NE**$(12,11)$: $(13,12), d_3 = 12 + -6 = 6$
- $d_3 > 0$: **NE**$(13,12)$: $(14,13), d_4 = 6 + -6 = 0$
- $d_4 > 0$: **NE**$(14,13)$: $(15,14), d_5 = 0 + -6 = -6$
- $d_5 < 0$: **E**$(15,14)$: $(16,14), d_6 = 14 + -6 = 8$
- $d_6 > 0$: **NE**$(16,14)$: $(17,15), d_7 = 8 + -6 = 2$
- $d_7 > 0$: **NE**$(17,15)$: $(18,16), d_8 = 2 + -6 = -4$
- $d_8 < 0$: **E**$(18,16)$: $(19,16), d_9 = -4 + 14 = 10$
- $d_9 > 0$: **NE**$(19,16)$: $(20,17)$

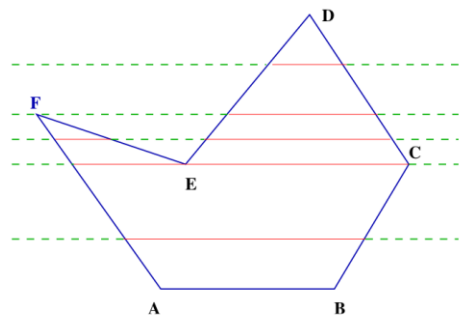
Filled Rectangles

- Write to all pixels within the rectangle.
- How about non-upright rectangles?
- General polygons?

Function FilledRectangle $(x_m, x_M, y_m, y_M,$ colour$)$
    for $x_m \le x \le x_M$ do
        for $y_m \le y \le y_M$ do
            WritePixel $(x, y.$ colour$)$
EndFunction

Scan Converting a Polygon

- For each scan line, compute all x where the line $y =$ *constant* meets polygon edges.
- Sort these x-values in increasing order.
- Take them in pairs $(x_1, x_2), (x_3, x_4), \ldots;$
- For each pair, fill pixels from x_1 to $x_2 - 1$, x_3 to $x_4 - 1$, etc. (interval closed on left, open on right) to avoid double-drawing shared edges.



Exploiting Edge Coherence

- If scan line y intersects with an edge E, it is likely that $y + 1$ also does. (Unless intersection is the y_{max} vertex.)
- When moving from y to $y + 1$, the X-coordinate goes from x to $x + 1/m$.
- $\frac{1}{m} = \frac{x_2 - x_1}{y_2 - y_1} = \frac{\Delta x}{\Delta y}$, store each fraction as integer
- For next scan line, add Δx to numerator. If sum goes > Δy, increment integer portion, subtract Δy from numerator.

- Let's take an edge from $(10, 5)$ to $(18, 20)$
- So $\Delta x = 18 - 10 = 8, \Delta y = 20 - 5 = 15$

i	y	x	Δx_n	Check $\Delta x_n > \Delta y$	Action	Δx_n
0	5	10	8	False	$x_1 = 10$	8
1	6	10	8+8	True	$x_2 = 10 + 1$	$16 - 15$
2	7	11	1+8	False	$x_3 = 11$	9
3	8	11	9+8	True	$x_4 = 12$	17-15
4	9	12	2+8	False	$x_5 = 12$	10
5	10	12	10+8	True	$x_6 = 12 + 1$	18-15
6	11	13	3+8	False	$x_7 = 13$	11
7	12	13	11+8	True	$x_8 = 13 + 1$	19-15
8	13	14	4+8	False	$x_9 = 14$	12
9	14	14	12+8	True	$x_{10} = 15$	20-15

- $(15,15)$ $(15,16)$ $(16,17)$ $(16,18)$ $(17,19)$ $(18,20)$

Scan Conversion Summary

- Filling the frame buffer given 2D primitives.
- Convert an analytical description of the basic primitives into pixels on an integer grid in the frame buffer.
- Can be performed for Lines, Polygons, Circles, etc. either filled or unfilled primitives.
- Efficient algorithms required since scan conversion is done repeatedly.
- Special hardware used these days 2D Scan Conversion is all, even for 3D graphics.

Fragment Processing

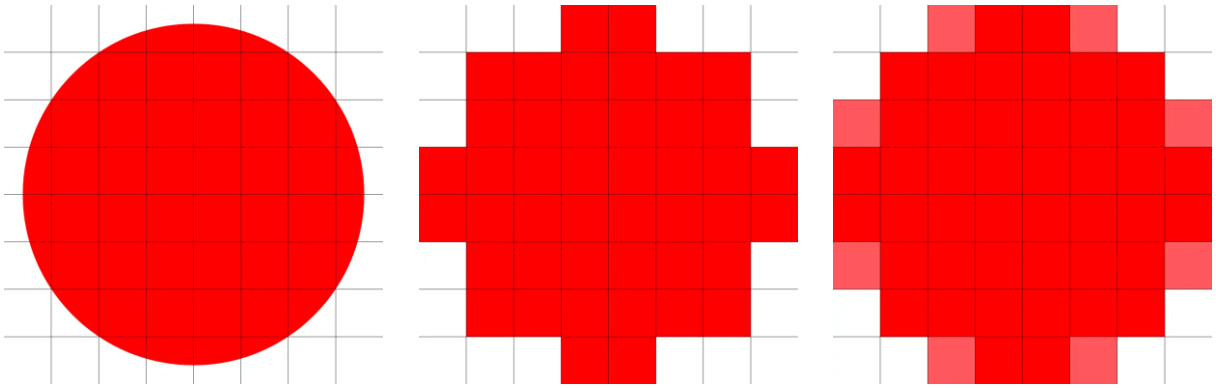
Fragment Processing

- The pixels generated by the rasterization stage are processed in arbitrary order by this stage.
- Depth value is available already. Can look up Z-buffer to keep or discard the fragment.
- Interpolated colour value can be sent to frame buffer.
- Texture image can be accessed using the texture coordinates. The final colour can be a combination of interpolated and texture colours.
- Modern GPUs: This stage is programmable! Can write own fragment shader to replace the standard processing.

Fragment Shader

1 Fragment per pixel

4 Fragment per pixel

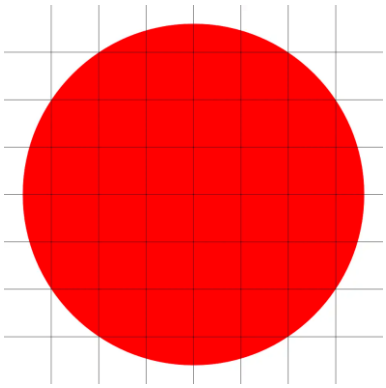


https://shader-tutorial.dev/basics/fragment-shader/

Fragment Shader

1 Fragment per pixel

4 Fragment per pixel



At a much smaller scale, the second result looks considerably more like a circle than the first result.

https://shader-tutorial.dev/basics/fragment-shader/

Thank You

Exploiting Edge Coherence

Relevant Data Structures

Edge Bucket (EB)

yMax	yMin	x	sign	dX	dY	sum

- **yMax:** Maximum Y position of the edge
- **yMin:** Minimum Y position of the edge
- **x:** The current x position along the scan line, initially starting at the same point as the **yMin** of the edge
- **sign:** The sign of the edge's slope (either -1 or 1)
- **dX:** The absolute delta x (difference) between the edge's vertex points
- **dY:** The absolute delta y (difference) between the edge's vertex points
- **sum:** Initiated to zero. Used as the scan lines are being filled to x to the next position

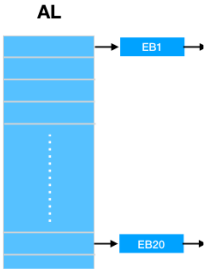
Relevant Data Structures

Edge Table (ET)

- When creating edges, the vertices of the edge need to be ordered from left to right
- The edges are maintained in increasing yMin order
- The edges are removed from the ET once the Active List is done processing them
- The algorithm is done filling the polygon once all of the edges are removed from the ET

Active edge List (AL)

- Edges are pushed into the AL from the Edge Table once an edge's yMin is equal to the current scan line being processed
- Edges will always be in the AL in pairs
- Edges in the AL are maintained in increasing x order.
- The AL will be re-sorted after every pass



Relevant Data Structures

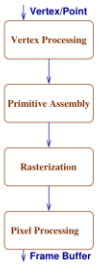
Polygon Scan Conversion Steps:

```
1. Create ET
    1. Process the vertices list in pairs, start with
[numOfVertices-1] and [0].
    2. For each vertex pair, create an edge bucket
2. Sort ET by yMin
3. Process the ET
    1. Start on the scan line equal to theyMin of the first
edge in the ET
    2. While the ET contains edges
        1. Check if any edges in the AL need to be removes
(when yMax == current scan line)
            1. If an edge is removed from the AL, remove
the associated the Edge Bucket from the Edge Table.
        2. If any edges have a yMin == current scan line,
add them to the AL
        3. Sort the edges in AL by X
        4. Fill in the scan line between pairs of edges in
AL
        5. Increment current scan line
        6. Increment all the X's in the AL edges based on
their slope
            1. If the edge's slope is vertical, the
bucket's x member is NOT incremented.
```

Recap

- Objects represented/approximated using geometric (1 D and 2D) primitives Primitives using (2D/3D) points in a natural coord frame Points transformed to screen coords in a few steps Primitives assembled and converted to pixels on screen Colour at each pixle: physics and interpolation Visibility evaluation to identify which is closer and farther Form image on framebuffer, which appears on the display

Recap



- **Vertex** stage: transform to screen coords, compute lighting in 3D
- Primitive assembly: form polygon/triangle/line
- Rasterization: Clip & Determine pixels inside each primitive
- **Pixel** stage: give colourto each pixel, perform Z-buffering

- Vertex stage: transform to screen coords, compute lighting in 3D
- Primitive assembly: form polygon/triangle/line
- Rasterization: Clip & Determine pixels inside each primitive
- Pixel stage: give colour to each pixel, perform Z-buffering